Experiment No. 09 — Implementation of Linear SVM

Counterfeit Banknote Detection using Linear SVM

Problem Statement

Develop a Linear Support Vector Machine (SVM) classification model that learns a separating hyperplane from labelled banknote samples and deploy it as a web application using Flask and HTML.

Objective

Use the Banknote Authentication dataset to predict whether a banknote is **genuine or counterfeit** from four numerical features extracted from its image:

- Variance
- Skewness
- Curtosis
- Entropy

Algorithm: Linear Support Vector Machine (SVM)
Dataset: `data_banknote_authentication.csv`

1. Import Libraries

Import the libraries required for data handling, visualization, model training, and evaluation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

2. Loading the Banknote Authentication Dataset from Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
file_path = "/content/drive/MyDrive/data_banknote_authentication.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully!
Dataset shape: (1372, 5)
```

	variance	skewness	curtosis	entropy	class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

3. Inspect the Dataset

The dataset contains four input features and one target column named `class`. In the commonly used dataset encoding, class `0` represents genuine banknotes and class `1` represents counterfeit banknotes.

```
print("Column names:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nClass distribution:")
print(df["class"].value_counts().sort_index())

print("\nTotal records:", len(df))
```

```
Column names:
['variance', 'skewness', 'curtosis', 'entropy', 'class']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   variance  1372 non-null   float64
 1   skewness  1372 non-null   float64
 2   curtosis  1372 non-null   float64
 3   entropy   1372 non-null   float64
 4   class     1372 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 53.7 KB

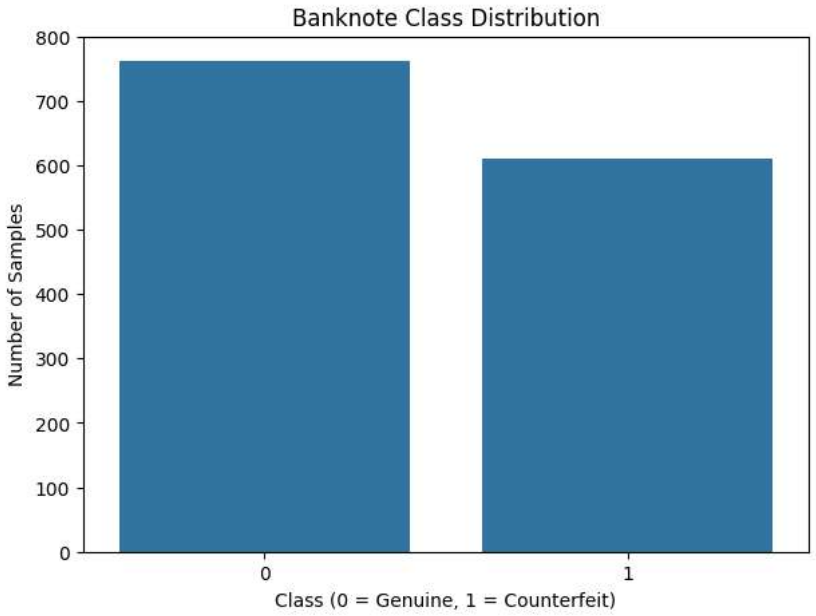
Missing values:
variance    0
skewness    0
curtosis    0
entropy     0
class       0
dtype: int64

Class distribution:
class
0    762
1    610
Name: count, dtype: int64

Total records: 1372
```

4. Visualize the Class Distribution

```
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="class")
plt.title("Banknote Class Distribution")
plt.xlabel("Class (0 = Genuine, 1 = Counterfeit)")
plt.ylabel("Number of Samples")
plt.show()
```



5. Prepare Features and Target

`X` contains the four numerical measurements. `y` contains the class label to predict.

```
feature_columns = ["variance", "skewness", "curtosis", "entropy"]

X = df[feature_columns]
y = df["class"].astype(int)

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
print("\nFeatures:")
print(feature_columns)
```

```
Feature shape: (1372, 4)
Target shape: (1372,)

Features:
['variance', 'skewness', 'curtosis', 'entropy']
```

6. Split the Dataset

Use 75% of the records for training and 25% for testing. `stratify=y` preserves the class proportions in both sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 1029
Testing samples: 343
```

7. Feature Scaling and Linear SVM

SVM is sensitive to feature scales, so use `StandardScaler`. A pipeline ensures scaling is fitted only on training data and applies the same transformation during prediction.

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="linear", C=1.0))
])

model.fit(X_train, y_train)

print("Linear SVM model trained successfully!")
```

```
Linear SVM model trained successfully!
```

8. Make Predictions

```
y_pred = model.predict(X_test)

results = pd.DataFrame({
    "Actual Class": y_test.to_numpy(),
    "Predicted Class": y_pred
})

display(results.head(10))
```

	Actual Class	Predicted Class
0	0	0
1	1	1
2	0	0
3	1	1
4	1	1
5	0	0
6	1	1
7	1	1
8	1	1
9	1	1

9. Evaluate the Model

Accuracy measures the percentage of correct predictions. The classification report includes precision, recall, and F1-score for each class.

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=["Genuine (0)", "Counterfeit (1)"]
))
```

```
Accuracy: 97.67%

Classification Report:
                precision    recall  f1-score   support

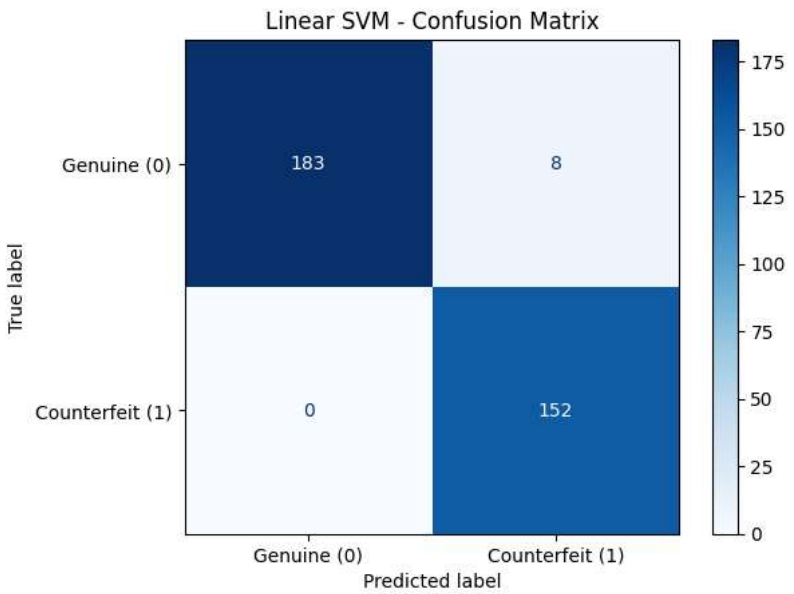
    Genuine (0)       1.00      0.96      0.98       191
Counterfeit (1)       0.95      1.00      0.97       152

       accuracy                           0.98       343
      macro avg       0.97      0.98      0.98       343
   weighted avg       0.98      0.98      0.98       343
```

10. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
```

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Genuine (0)", "Counterfeit (1)"]
)
disp.plot(cmap="Blues")
plt.title("Linear SVM - Confusion Matrix")
plt.show()
```



11. Predict a New Banknote Sample

Enter four numerical feature values in this order: variance, skewness, curtosis, entropy. The example values below are only an example input; replace them with values you want to test.

```
# Example feature values: variance, skewness, curtosis, entropy
sample = pd.DataFrame(
    [[3.6216, 8.6661, -2.8073, -0.44699]],
    columns=feature_columns
)

prediction = int(model.predict(sample)[0])

if prediction == 0:
    print("Prediction: Genuine banknote")
else:
    print("Prediction: Counterfeit banknote")

print("Predicted class:", prediction)
```

```
Prediction: Genuine banknote
Predicted class: 0
```

12. Save the Trained Model

The saved pipeline includes both the scaler and the Linear SVM classifier. The Flask application can load this single file to make predictions.

```
joblib.dump(model, "linear_svm_banknote_model.pkl")
print("Saved: linear_svm_banknote_model.pkl")
```

```
Saved: linear_svm_banknote_model.pkl
```

13. Download the Trained Model

Run this cell to download the model file to your computer. Place it in the same folder as your Flask application when you build the web interface.

```
from google.colab import files
files.download("linear_svm_banknote_model.pkl")
```

Flask Code :-

```
from flask import Flask, render_template, request
import joblib
import numpy as np

app = Flask(__name__)

# Load the trained Linear SVM pipeline
model = joblib.load("linear_svm_banknote_model.pkl")


@app.route("/", methods=["GET", "POST"])
def home():
    prediction = None
    error = None
    values = {
        "variance": "",
        "skewness": "",
        "curtosis": "",
        "entropy": ""
    }

    if request.method == "POST":
        try:
            # Read the four banknote features
            for feature in values:
                values[feature] = request.form.get(feature, "").strip()

            if any(value == "" for value in values.values()):
                raise ValueError("Please enter all four feature values.")

            features = np.array([[
                float(values["variance"]),
                float(values["skewness"]),
                float(values["curtosis"]),
                float(values["entropy"])
            ]])

            if not np.isfinite(features).all():
                raise ValueError("Please enter valid finite numbers.")

            # Predict using the trained model
            result = int(model.predict(features)[0])

            if result == 0:
                prediction = {
                    "label": "Genuine Banknote",
                    "description": "The model classifies this sample as genuine.",
                    "category": "genuine"
                }
            else:
                prediction = {
                    "label": "Counterfeit Banknote",
                    "description": "The model classifies this sample as counterfeit.",
                    "category": "counterfeit"
                }

        except ValueError as exc:
            error = str(exc) or "Please enter valid numerical values."
        except Exception:
            error = "Unable to classify the sample. Check the model file and input values."

    return render_template(
        "index.html",
```

```
            prediction=prediction,
            error=error,
            values=values
        )


    if __name__ == "__main__":
        app.run(debug=True)
```

HTML Code :-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Banknote Authentication | Linear SVM</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background: #f1f5f9;
            color: #1e293b;
        }

        header {
            background: #1e3a8a;
            color: white;
            text-align: center;
            padding: 30px 15px;
        }

        header h1 {
            margin: 0 0 10px;
            font-size: 29px;
        }

        header p {
            margin: 0;
            color: #dbeafe;
        }

        .container {
            max-width: 760px;
            width: 92%;
            margin: 30px auto;
        }

        .card {
            background: white;
            padding: 28px;
            margin-bottom: 22px;
            border-radius: 14px;
            box-shadow: 0 5px 18px rgba(0, 0, 0, 0.07);
        }

        h2 {
            color: #1e3a8a;
            margin-top: 0;
        }

        .intro {
            line-height: 1.7;
            color: #475569;
        }

        .form-grid {
            display: grid;
```

```
            grid-template-columns: 1fr 1fr;
            gap: 18px;
        }

        label {
            display: block;
            font-weight: bold;
            margin-bottom: 8px;
        }

        input {
            width: 100%;
            padding: 12px;
            border: 1px solid #cbd5e1;
            border-radius: 7px;
            font-size: 15px;
        }

        input:focus {
            outline: 2px solid #93c5fd;
            border-color: #2563eb;
        }

        button {
            width: 100%;
            margin-top: 24px;
            padding: 14px;
            background: #1e3a8a;
            color: white;
            border: none;
            border-radius: 8px;
            font-size: 16px;
            font-weight: bold;
            cursor: pointer;
        }

        button:hover {
            background: #1d4ed8;
        }

        .result {
            text-align: center;
            border-top: 5px solid #16a34a;
        }

        .result.genuine {
            border-top-color: #16a34a;
        }

        .result.counterfeit {
            border-top-color: #dc2626;
        }

        .result-label {
            display: inline-block;
            padding: 12px 20px;
            border-radius: 8px;
            font-size: 21px;
            font-weight: bold;
        }

        .genuine .result-label {
            background: #dcfce7;
            color: #166534;
        }

        .counterfeit .result-label {
            background: #fee2e2;
            color: #991b1b;
        }

        .error {
            background: #fee2e2;
            color: #991b1b;
            padding: 14px;
            border-radius: 8px;
            margin-bottom: 20px;
        }
```

```
        .note {
            color: #64748b;
            font-size: 13px;
            line-height: 1.6;
            margin-top: 18px;
        }

        footer {
            text-align: center;
            padding: 22px;
            color: #64748b;
            font-size: 14px;
        }

        @media (max-width: 550px) {
            .form-grid {
                grid-template-columns: 1fr;
            }

            .card {
                padding: 20px;
            }

            header h1 {
                font-size: 24px;
            }
        }
    </style>
</head>

<body>

<header>
    <h1>Banknote Authentication</h1>
    <p>Counterfeit Banknote Detection using Linear SVM</p>
</header>

<main class="container">

    <section class="card">
        <h2>About the Project</h2>
        <p class="intro">
            Enter the four numerical features extracted from a banknote image.
            The trained Linear Support Vector Machine (SVM) will classify
            the sample as genuine or counterfeit.
        </p>
    </section>

    <section class="card">
        <h2>Enter Banknote Features</h2>

        {% if error %}
            <div class="error">{{ error }}</div>
        {% endif %}

        <form method="POST">
            <div class="form-grid">

                <div>
                    <label for="variance">Variance</label>
                    <input
                        type="number"
                        step="any"
                        id="variance"
                        name="variance"
                        value="{{ values.variance }}"
                        placeholder="Enter variance"
                        required>
                </div>

                <div>
                    <label for="skewness">Skewness</label>
                    <input
                        type="number"
                        step="any"
                        id="skewness"
```

```
                        name="skewness"
                        value="{{ values.skewness }}"
                        placeholder="Enter skewness"
                        required>
                </div>

                <div>
                    <label for="curtosis">Curtosis</label>
                    <input
                        type="number"
                        step="any"
                        id="curtosis"
                        name="curtosis"
                        value="{{ values.curtosis }}"
                        placeholder="Enter curtosis"
                        required>
                </div>

                <div>
                    <label for="entropy">Entropy</label>
                    <input
                        type="number"
                        step="any"
                        id="entropy"
                        name="entropy"
                        value="{{ values.entropy }}"
                        placeholder="Enter entropy"
                        required>
                </div>

            </div>

            <button type="submit">Classify Banknote</button>
        </form>

        <p class="note">
            Enter values obtained from the dataset or from extracted banknote
            image features. The model uses the same feature order as its training.
        </p>
    </section>

    {% if prediction %}
    <section class="card result {{ prediction.category }}">
        <h2>Prediction Result</h2>

        <div class="result-label">
            {{ prediction.label }}
        </div>

        <p>{{ prediction.description }}</p>

        <p class="note">
            This is a machine-learning prediction, not a guarantee of authenticity.
        </p>
    </section>
    {% endif %}

</main>

<footer>
    Experiment No. 09 | Linear SVM Classification
</footer>

</body>
</html>
```